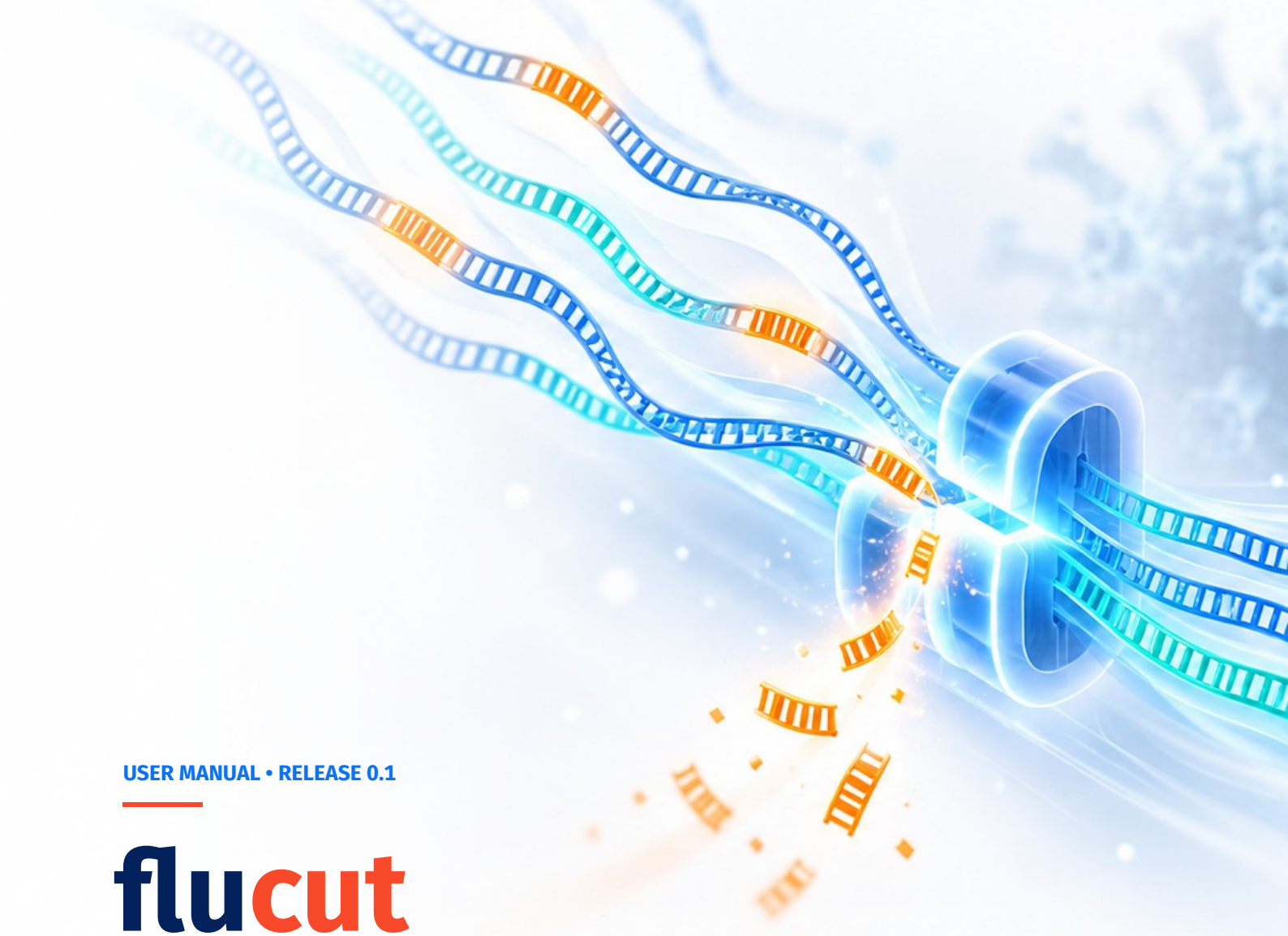


USER MANUAL • RELEASE 0.1

flucut

Primer trimming for influenza A before the assembly with IRMA

Uni12/Uni13 amplicons, internal priming

Giorgio Gonnella
BAIA, Institut Pasteur du Cambodge

October 30, 2026

Contents

1 Introduction **3**
- 1.1 What flucut does . . . 3
- 1.2 Why it is needed . . . 3
- 1.3 What it does not do . . . 4
- 1.4 Platforms . . . 4

2 Installation **5**
- 2.1 From the source . . . 5
- 2.2 The tests . . . 5
- 2.3 Packages . . . 5

3 Quick start **7**

4 The primers **9**
- 4.1 The conserved termini . . . 9
- 4.2 The built-in primers . . . 9
- 4.3 The schemes . . . 10

5 How it works **11**
- 5.1 The layout of a read . . . 11
- 5.2 The search of a primer . . . 11
- 5.3 The window . . . 12
- 5.4 The adapters . . . 12
- 5.5 The trimming . . . 13
- 5.6 Threads . . . 13

6 Internal priming **15**
- 6.1 The artefact . . . 15
- 6.2 What flucut does about it . . . 15
- 6.3 Classification against references . . . 16
- 6.4 The length check . . . 16

7 Outputs **17**
- 7.1 The trimmed reads . . . 17
- 7.2 summary.tsv . . . 18
- 7.3 lengths.tsv . . . 18

7.4 flucut_report.html, flucut_summary.json . . . 19
7.5 Messages . . . 19

8 Reports **21**
8.1 The HTML report . . . 21
8.2 The JSON summary . . . 23

9 Live mode **27**
9.1 How the files are taken . . . 27
9.2 The reports . . . 27
9.3 The end . . . 28
9.4 Resuming . . . 28

10 Using the reads with IRMA **29**
10.1 What was checked . . . 29

11 Performance and accuracy **31**
11.1 The consensus . . . 31
11.2 The reads . . . 31
11.3 Reads with barcodes . . . 32
11.4 Speed . . . 32

12 Troubleshooting **33**
12.1 Errors in the command line . . . 33
12.2 Errors in the inputs . . . 33
12.3 Messages about the reads . . . 34

13 Command line reference **35**
13.1 Arguments . . . 35
13.2 Options . . . 35
13.3 Classification against the references . . . 36
13.4 Live mode . . . 36
13.5 Other options . . . 37

14 Glossary **39**

15 About **41**
15.1 Use of AI tools . . . 41

flucut removes the primers of the influenza A multisegment RT-PCR (the universal Uni12 and Uni13 primers) from sequencing reads, before the assembly of the segments, e.g. with IRMA. It finds the primers at both ends of each read, in both orientations, also when a read begins inside a primer, and removes them by position. It recognises the reads started by a primer at an internal site of a segment (internal priming), whose primer bases otherwise reach the consensus as false variants, such as the I348K change of the H5 hemagglutinin.

This manual explains how to install and use flucut, from a first run on the test data of the repository to the classification of the reads against reference sequences. The chapters can be read in order, as a tutorial, or used as a reference: the last chapters list every message and every option.

1 Introduction

1.1 What flucut does

The whole genome of influenza A is often sequenced from a multisegment RT-PCR: two universal primers, Uni12 and Uni13, bind the conserved termini of the eight segments and amplify all of them in one reaction. The bases of a read which lie under a primer come from the primer, not from the virus: a difference between the virus and the primer at such a position is overwritten. flucut removes these bases from the reads, before the assembly.

For each read, flucut:

- finds the primers at both ends of the read, in either orientation, with sequencing errors, also when the read begins or ends inside a primer;
- removes every primer-derived base and everything outside the primers (adapter, barcode and other flanking bases), by position;
- writes the rest of the read, in its original orientation, to a gzipped FASTQ file for each sample, with tags in the title telling what was found and removed;
- with reference sequences of the segments, tells whether each end of the read is at a terminus of a segment or at an interior position (internal priming), and reports it.

1.2 Why it is needed

The untailed Uni12 primer is only twelve bases long, with a degenerate base. Sequences similar enough to anneal to it occur inside the segments: the primer can start an amplicon in the middle of a segment, and the reads of such an amplicon carry the primer's bases where the template has others. In the H5 hemagglutinin (HA) of Cambodian H5N1 viruses this changes the isoleucine at position 348 to a lysine (I348K): a false variant, reproducible from run to run (see *Internal priming*). Removing the primers from the reads removes it.

IRMA, the assembler used for influenza by many laboratories, does not remove primers: it matches a single adapter sequence, and its pipeline MIRA does no primer trimming for influenza Nanopore reads.

1.3 What it does not do

- It does not assign the reads to samples (demultiplexing): give it the reads of one sample (a file, or a directory of files, e.g. a barcode directory written by the basecaller).
- It does not assemble the segments or call variants: that is the work of IRMA or another assembler.
- It knows the primers of the influenza A multisegment RT-PCR (*The primers*); other amplicon schemes are not supported.

1.4 Platforms

flucut is not specific to a sequencing platform: the errors allowed in a primer are an option. The defaults are those of Oxford Nanopore reads.

2 Installation

2.1 From the source

Only a C compiler (C11 with GNU extensions) and `make` are needed: the libraries used (libcbaia, edlib, libyaml, zlib) are included in the repository, in `ext`.

```
git clone https://github.com/baia-ipc/flucut
cd flucut
make
make test
```

The program is `bin/flucut`. `make install` copies it to `/usr/local/bin` (`make install PREFIX=<dir>` to another prefix).

`make SYSTEM_LIBS=1` uses the zlib and libyaml of the system instead of the included copies.

2.2 The tests

`make test` runs the unit tests and the program tests under valgrind (if it is installed), and the check of the coding style. The program tests run flucut on the files of `testdata` and compare the results with `testdata/expected`.

2.3 Packages

There is no Bioconda package yet.

3 Quick start

The repository contains a small test set in `testdata`: twelve reads simulated from a real HA record (NCBI `NC_004908.1`), each one a typical situation (see `testdata/README.md`). From the root of the repository, after `make`:

```
bin/flucut -o out testdata/reads.fastq
```

flucut writes:

```
SUCCESS | out/reads.fastq.gz: 11 of 12 reads written (10 with both
  primers, 1 with one, 1 with none; 1 too short, of which 1 dimers)
SUCCESS | Reports written: out/flucut_report.html,
  out/flucut_summary.json
```

(the lines are wrapped here, without the time stamps). The output directory contains:

`reads.fastq.gz`
: the trimmed reads, ready for IRMA;

`summary.tsv`
: the counts of the reads;

`lengths.tsv`
: the lengths of the complete amplicons.

The first read, an amplicon with both primers, became:

```
@full_plus simulated  fs:Z:+  fl:Z:uni12r  fr:Z:mbtuni-13  tl:i:12  tr:i:23
GGAATTACTTAACTAGCAAAATGGAAACAATATCACTAATAACTATACTACTAG...
```

The tags tell that the read is of the plus strand of the amplicon (`fs`), that the primer `uni12r` was found at its left end and `mbtuni-13` at its right end, and that 12 and 23 bases were removed at its start and end (*Outputs*). The primer dimer (two primers and nothing between them) was not written.

With the reference of the segment, flucut also tells where each read started:

```
bin/flucut -o out2 -r testdata/ref-ha.fasta testdata/reads.fastq
```

It reports the ends started by internal priming and the complete amplicons much shorter than the reference (the figure shows this run in a terminal, where flucut shows its banner first), and the read simulated as started inside the segment, at position 712, is tagged as such:

```
@internal12_plus simulated  fs:Z:+  fl:Z:uni12r  ol:Z:internal
  ml:Z:NC_004908.1:712:+  fr:Z:mbtuni-13  or:Z:terminal
  mr:Z:NC_004908.1:1714:-  tl:i:12  tr:i:23
```

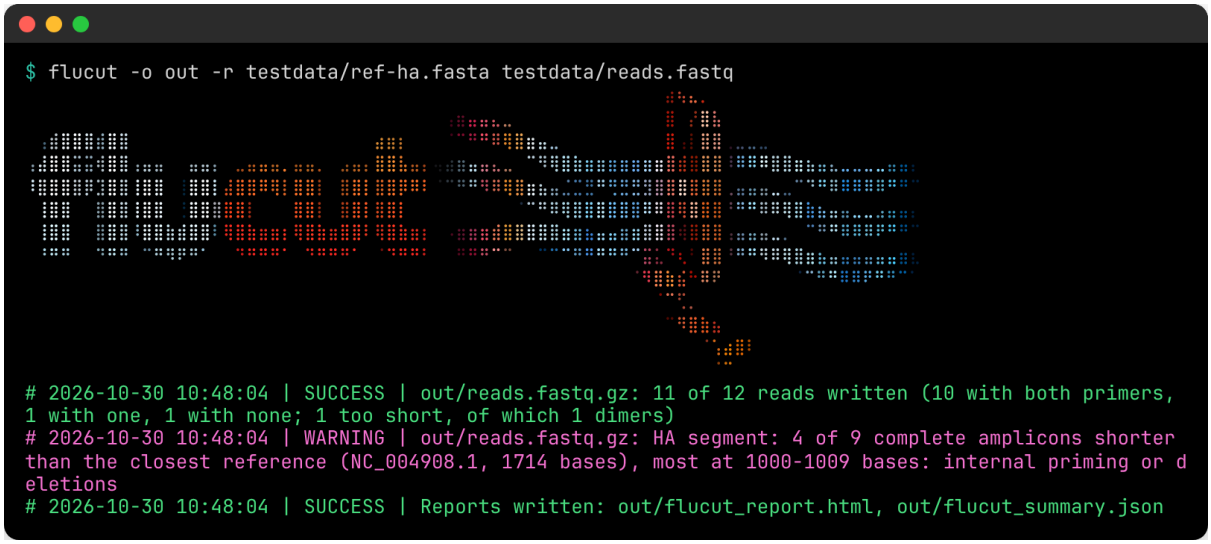


Fig. 1: `flucut -o out -r testdata/ref-ha.fasta testdata/reads.fastq` in a terminal of 100 columns (`terminal.js`).

To trim a real run, give the reads of each sample, as files or as directories (one output per input):

```
flucut -o trimmed barcode01/ barcode02/ barcode03/
```

or the `fastq_pass` directory of the run, which gives one output for each of its barcode directories:

```
flucut -o trimmed run/fastq_pass/
```

and give the outputs to IRMA (*Using the reads with IRMA*).

4 The primers

4.1 The conserved termini

All influenza A segments end in the same conserved sequences: the 12 nucleotides of one end and the 13 of the other are (almost) identical in all eight segments and in all subtypes. On the plus strand of an amplicon (the orientation of the mRNA) they are:

```
AGCAAAAGCAGG ... segment ... CCTTGTTTCTACT
```

The universal primers bind them: Uni12 (`AGCAAAAGCAGG`) at the left end, Uni13 (`AGTAGAAACAAGG`, the reverse complement of the right end) at the right end. The fourth base of the 12-nucleotide terminus is A in some segments and G in others; the degenerate form of Uni12, `AGCRAAAGCAGG` (R = A or G), matches both.

4.2 The built-in primers

flucut knows the primers by name; their sequences are quoted from the publications, each one documented in `src/primers.h`.

name	sequence	source
uni12	`AGCAAAAGCAGG`	the conserved terminus (Uni12 of Hoffmann et al. 2001, as quoted by Zhou et al. 2009)
uni12r	`AGCRAAAGCAGG`	the same, degenerate fourth base
uni13	`AGTAGAAACAAGG`	the conserved terminus (Uni13)
mbtuni-12	`ACGCGTGATCAGCAAAAGCAGG`	Zhou et al. 2009, J Virol 83:10309-10313
mbtuni-12.4r	`ACGCGTGATCAGCRAAAGCAGG`	Licheri et al. 2025, Genome Med 17:103
mbtuni-13	`ACGCGTGATCAGTAGAAACAAGG`	Zhou et al. 2009

The tailed primers carry the 5' tail `ACGCGTGATCAG`, which overlaps the terminus by two bases: flucut stores each primer whole, with the position at which its conserved terminus begins, and knows which bases of a read come from the tail and which from the terminus.

4.3 The schemes

A scheme names the primers expected at each end of an amplicon (option `-s`):

scheme	left end (Uni12 side)	right end (Uni13)
uni12-mbt13	uni12r	mbtuni-13
mbt	mbtuni-12, mbtuni-12.4r	mbtuni-13
uni	uni12, uni12r	uni13

`uni12-mbt13` is the default: the untailed degenerate Uni12 primer and the tailed Uni13 primer, the combination used at the Institut Pasteur du Cambodge. When a scheme has several primers at an end, the best match is taken.

5 How it works

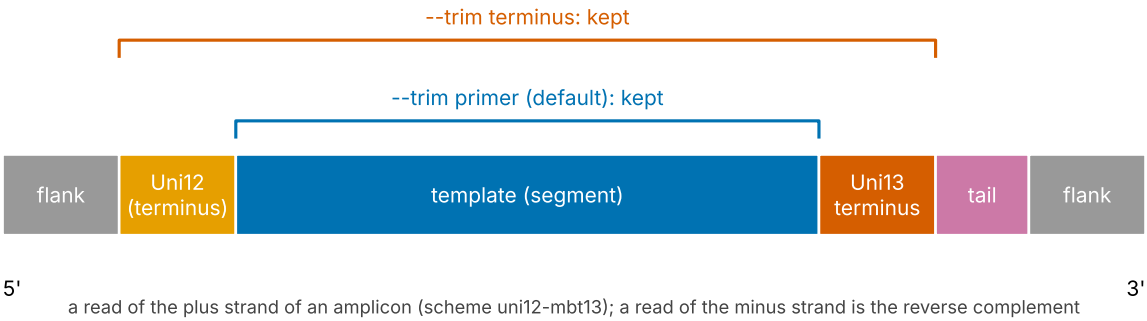


Fig. 1: The parts of a read of an amplicon, and what each trimming policy keeps (`make_diagrams.py`).

5.1 The layout of a read

A read of an amplicon contains, from one end to the other: flanking bases (adapter, barcode, or bases of the library preparation), the primer of one end, the template (the segment), the primer of the other end, and flanking bases again. It can be sequenced in either orientation, and it can begin or end inside a primer.

flucut searches the primers of the left end of the scheme at the start of the read and those of the right end at its end. If the read was sequenced in the other orientation, the primers are found in its reverse complement; the positions are always reported in the original read, which is not reverse-complemented.

5.2 The search of a primer

A primer is aligned to the read with edit distance (substitutions, insertions and deletions), allowing `-e` errors (a fraction of the length of the primer: with 0.2, 2 errors in Uni12). Degenerate bases (e.g. the R of Uni12) match any of the bases they stand for, in the primer and in the read (e.g. N).

At an end of the read, the primer can be partial:

- it can overhang the read, when the read begins inside the primer: the missing part is not counted as errors, and up to half of the primer can be missing;

- it can lack up to half of its outer bases after some flanking bases, when the amplicon was shortened before the adapter was ligated; this shorter form must match with one error less than its length allows, as a shorter sequence matches by chance more easily.

The whole primer and its shorter forms are searched in one alignment, in which the missing outer bases cost as many errors as the shorter form is allowed fewer: the match with the fewest errors, counting this cost, is taken, and among equally good matches the outermost. The overhang is considered only if the primer is not found inside the read.

With `-e auto` (the default), the errors allowed are decided from the first 1000 reads of the first input file: the primers are searched in them with 0.2, and the error rate of their alignments (errors per aligned read base) is measured. Below 4%, 0.2 is used; from 4% on, 0.3 (3 errors in Uni12, 6 in the tailed Uni13). On reads simulated with mean error rates of 1, 2, 4, 8 and 12% (measured: 0.9, 1.7, 3.0, 5.0 and 6.3%: the alignments with more errors than allowed are not found), 0.2 left few primer bases up to 4%; at 8 and 12%, 0.3 left 4 times fewer primer bases and 2 to 4 times fewer bases of the internal priming site than 0.2, at the cost of about 3 times more template bases removed (`benchmark/RESULTS.md`). As in Lumipore, the rule has two values and one threshold, measured once, before the trimming.

5.3 The window

A match of a primer deep inside a read is not the primer which started the amplicon: trimming there would remove template. The primer of an end is therefore accepted only if it begins (left end) or ends (right end) within `-w` bases of that end of the read (default 100: enough for the barcoding adapters of Oxford Nanopore, 46 bases, and further flanking bases).

Within the window, the outermost of the best matches of a primer is taken, and the outermost of the matches of the primers of an end (e.g. the two forms of a degenerate primer): the primer which started the amplicon is the one nearest to the end of the read, and a primer-like sequence further inside is template. Matches of a primer between the two end primers are counted (tag `ni`), not trimmed: they are the sites at which a primer can anneal, and the junctions of chimeras.

5.4 The adapters

At an end of the read at which no primer is found (the molecule was broken, or the read begins too far inside the primer), flucut searches the sequencing adapters of Oxford Nanopore (`--adapters`, default: all the kits): the native barcoding adapters (`nb`, e.g. SQK-NBD114), the rapid barcoding adapter (`rbk`, e.g. SQK-RBK114) and the ligation adapter (`lsk`, e.g. SQK-LSK114), with the sequences of Dorado. The barcode is searched as 24 Ns, which match any base: any barcode of the kit is recognised, none is identified (the reads normally come already demultiplexed). The errors allowed are 20% of the fixed bases of an adapter (e.g. 4 of the 22 of a native barcoding adapter); an adapter without a barcode may overhang the read by up to half of its length, one with a barcode may not (the part left in the read could then be mostly the barcode, which matches anything). The adapter must lie in the window of its end; of the adapters found, the innermost is taken (the barcode adapter, not the ligation adapter before it).

An adapter found is removed with everything outside it, and the part of the read between the adapters is analyzed again: a primer of which only a few bases are left is now at an end, next to the adapter, where it can overhang (up to half of it missing), and is removed as well. In this second analysis, the primer must lie within 5 bases of the adapter: in the usual window of the new end, a primer-like sequence of the template could otherwise be taken for it. The tags `a5` and `a3` tell which kit's adapter was removed at the start and at the end of a read.

The search uses the adaptor search of Lumipore (`src/adaptor-search.c`, unchanged); the adapters are searched only at the ends without a primer, so the reads with both primers are trimmed as without it.

5.5 The trimming

The removal is by position: the whole span of the primer alignment is removed, whether or not the bases under it differ from the primer, and everything outside it.

`--trim primer` (the default)
: removes every primer-derived base, tail and conserved terminus. No base of the output comes from a primer. The consensus is shorter than the segment by the termini (12 and 13 bases): those bases would have been the primer's, not measured sequence.

`--trim terminus`
: removes the tail and everything outside the primer, and keeps the bases of the conserved terminus, so that the consensus has complete ends. The kept bases are those of the primer: with the degenerate Uni12, the fourth base is a mix of A and G; where the segment differs from the primer (e.g. the 3' end of segment 2 of A/Puerto Rico/8/1934, `CCTTGTTCCTACT`), the primer's base is kept. At an internal priming site, the primer bases are not the template at all: this policy therefore requires the references (`-r`), against which such ends are recognised and trimmed whole (*Internal priming*).

Reads with nothing left (primer dimers) and reads shorter than `-m` bases after the trimming are not written.

5.6 Threads

The files are trimmed by `-j` worker threads, started for each file (as in Lumipore): each thread reads a batch of 1000 reads in turn, trims it and compresses it with its own objects, and writes it in the order of the input, waiting for its turn. Each batch is a gzip member of its own, so that the output file is the same, byte for byte, for any number of threads. Each thread counts its reads; the counts are summed at the end of the file.

6 Internal priming

6.1 The artefact

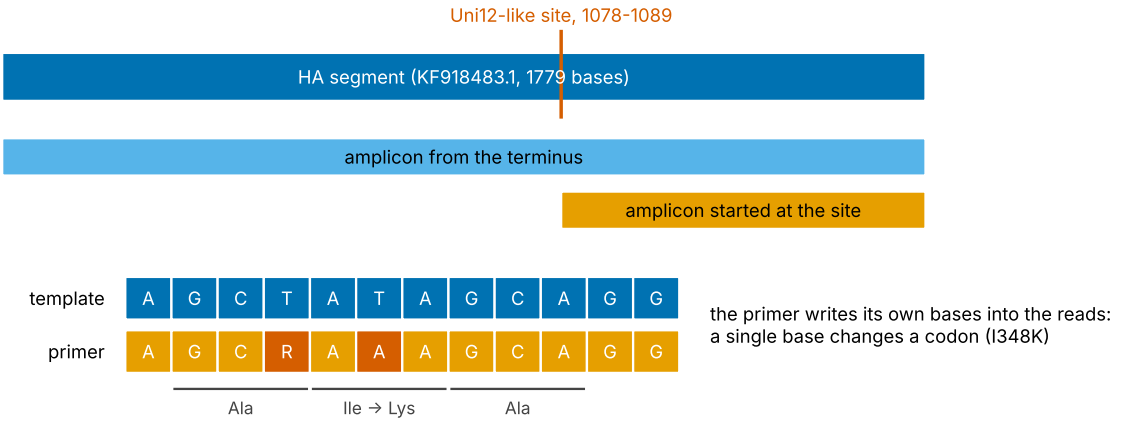


Fig. 1: The Uni12-like site of the HA of A/chicken/Cambodia/X0815302/2013 (H5N1), NCBI KF918483.1, and the amplicon the primer starts there (`make_diagrams.py`).

The untailed Uni12 primer is twelve bases long, with a degenerate base. A sequence similar enough to it occurs inside the HA segment of the H5N1 viruses of Cambodia: after the codons of the motif GLFGAIAG, the bases `AGCTATAGCAGG` differ from the primer `AGCRAAAGCAGG` at two positions. The primer can anneal there and start an amplicon in the middle of the segment. Its reads begin with the bases of the primer: the R falls on the third base of an alanine codon (silent), but the A replaces the T of the isoleucine codon ATA, which becomes the lysine codon AAA. If enough reads carry it, the consensus shows I348K, a variant which looks real and is not. All the 30 Cambodian H5N1 HA sequences of GenBank checked for the benchmark have this site.

6.2 What flucut does about it

The primer bases of an internally primed read are at its end, like those of any other read: removing every primer (`--trim primer`, the default) removes the artefact, without any reference. The site is still covered by the reads of the complete amplicons, which carry the template.

In the benchmark (*Performance and accuracy*), the untrimmed simulated reads give AAA at the site in

the IRMA consensus, and the reads trimmed by flucut give ATA, the truth.

6.3 Classification against references

With reference sequences of the segments (`-r`, a FASTA file), flucut also tells where each read started. The bases which follow the primer of an end (the probe: `--probe-len`, default 32 bases; for the right end, the reverse complement of the bases before its primer) are located in the references:

- at a terminus of a segment (within `--terminal-window` bases of it, default 60, as references often lack some bases of the termini): a normal amplicon (tags `ol`/`or`: `terminal`);
- at an interior position: internal priming (`internal`); the tags `ml`/`mr` give the reference and the position;
- not located (no reference of the subtype, a probe too short): `unknown`.

Among equally good locations, the one at a terminus is taken: an ambiguous read is never classified as internally primed.

The references are indexed by their k-mers (10-mers), so that the time hardly depends on their number; a probe without a candidate is aligned to all of them. `--exhaustive-search` aligns every probe to all of them, to check the index (the outputs were identical in the benchmark).

With `--trim primer` the classification is reported only. With `--trim terminus`, an end classified as internal loses its whole primer, and the others keep their terminus.

`scripts/fetch_references.py` downloads segment references from NCBI, with their provenance.

6.4 The length check

The complete amplicons of a segment (both primers found) have the length of the segment. Internal priming, or a deletion (e.g. in the defective interfering RNAs of the polymerase segments), shows as a group of shorter amplicons. flucut writes their lengths to `lengths.tsv` (*Outputs*):

- with references, for each reference the amplicons shorter than 90% of its length are counted (`short_complete` in `summary.tsv`) and reported, with the length where most of them are. The reference length is the measure, not the most frequent length, as the internally primed amplicons can be the majority;
- without references, the peaks of the lengths are listed.

In the benchmark, 209 of the 340 complete HA amplicons were reported as short, most at 670-679 bases: the length from the internal site to the end of the segment.

7 Outputs

The inputs are read as groups of files, each of which gives one output, `<outdir>/<name>.fastq.gz`:

- a file of reads given directly is a group, named as the file without its suffix (`.fastq`, `.fq`, optionally `.gz`, or `.bam`);
- a directory is a group of its files of reads, named as the directory;
- unless the directory contains subdirectories whose name starts with `barcode` (the `fastq_pass` directory of a MinKNOW run, with its `barcodeNN` directories): then each of them is a group, named as the subdirectory; the files beside them are not read (a warning is given), and the other subdirectories (e.g. `unclassified`) are ignored.

The files of a directory are read in the order of their names. Two groups with the same name are an error, as one would overwrite the other. `summary.tsv` and `lengths.tsv` have one part for each group.

The output directory also contains the reports of the run (*Reports*): `flucut_report.html`, the HTML report, and `flucut_summary.json`, the same content for programs, with the statistics of all reads and of each group.

7.1 The trimmed reads

`<name>.fastq.gz` (gzip, FASTQ) contains the reads which are not empty and not shorter than `-m` bases after the trimming, in the order of the input, in their original orientation, with their quality values cut as the sequence. Their title is the original title followed by tags in the SAM text format, separated by tabs (as in the FASTQ written by Dorado), unless `--no-tags` is given:

tag	meaning
`fs`	strand of the amplicon: `+`, `-`, or `.` (no primer)
`fl`	the primer found at the left end (Uni12 side)
`fr`	the primer found at the right end (Uni13 side)
`tl`	bases removed at the start of the read
`tr`	bases removed at the end of the read
`ni`	primer matches inside the read (if any)
`a5`	the kit of the adapter removed at the start of the read (`nb`, `rbk` or `lsk`; only at an end without a primer, see *How it works*)
`a3`	the same, at the end of the read
`ol`	with `-r`: origin of the left end (`terminal`, `internal` or `unknown`)
`or`	the same for the right end
`ml`	with `-r`: where the probe of the left end mapped, `<reference>:<position>:<strand>` (1-based position on the plus strand)
`mr`	the same for the right end

The left and right ends are those of the amplicon: for a read of the minus strand, the left end is at the end of the read. IRMA accepts the tags (*Using the reads with IRMA*).

7.2 summary.tsv

One line for each input, tab-separated, with a header:

column	meaning
`output`	the name of the input
`reads_in`	reads read
`bases_in`	their bases
`reads_out`	reads written
`bases_out`	their bases
`both_ends`	reads with the primers of both ends
`one_end`	reads with the primer of one end
`no_end`	reads without primers
`minus_strand`	reads found on the minus strand of the amplicon
`dimers`	primer dimers (both primers, nothing between)
`too_short`	reads not written: empty or shorter than `-m` (dimers included)
`with_internal_match`	reads with primer matches inside them
`left_terminal`, `left_internal`, `left_unknown`, `right_terminal`, `right_internal`, `right_unknown`	with `-r`: the left and right ends found, by origin; `NA` without references
`short_complete`	with `-r`: complete amplicons shorter than 90% of their reference; `NA` without references

7.3 lengths.tsv

The lengths of the complete amplicons (both primers found; the template between the two primers), in bins of 10 bases, tab-separated, with a header: `output`, `group` (the reference to which the probe of the left end, or else of the right end, mapped; `not_located`; or `all` without references), `length` (the start

of the bin) and `reads`. Only the bins with reads are written; the last bin collects the lengths from 4000 bases on.

7.4 flucut_report.html, flucut_summary.json

The reports of the run, always written, at its end: the run parameters and, for all reads and for each group, the reads at each step, the primers found at each end, the internal priming sites (with references) and the complete amplicons, with the histograms of their lengths (*Reports*). Each is written to a temporary file (suffix `.tmp`) and then renamed.

7.5 Messages

The messages go to the standard error (or to the file of `-L`): the counts of each input, the internal priming found, and the length check (*Internal priming*). In a terminal, the banner of flucut is shown first (not with `-q` or `-L`).

8 Reports

Besides the trimmed reads, the tables and the messages, flucut always writes two reports to the output directory:

- `flucut_report.html`, a report for people;
- `flucut_summary.json`, the same information for programs (pipelines, dashboards).

Both are written at the end of the run, to a temporary file which is then renamed, so that a partial report is never read. The report and the summary, their templates and plots come from Lumipore, with the content of flucut.

8.1 The HTML report

The report is a single, self-contained file: the plots are inline SVG, and it uses no external resources (and no JavaScript), so that it can be opened offline in any browser, archived, and sent by e-mail. It follows the light or dark theme of the browser.

The figures of this chapter show the report of the simulated reads of the benchmark (3,200 reads of the eight segments, of which 239 HA reads internally primed, *Performance and accuracy*), trimmed with the references (`-r`); see `docs/manual/figures/README.md` for how they were made.

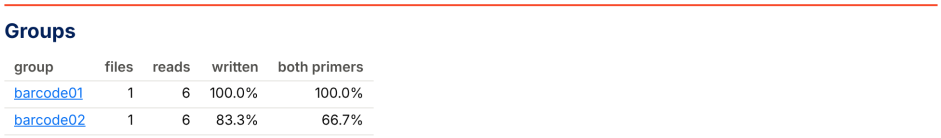
The run parameters.
: At the right of the logo: the version of flucut, the command line, the primer scheme, the trimming policy, the references and the options of the trimming.



version	`0.1`
command	`flucut -q -o out -r genome.fasta reads.fastq.gz`
primer scheme	`uni12-mbt13`
trimming	`primer (every primer-derived base removed)`
references	`genome.fasta (8 sequences)`
errors in a primer	`0.2 of its length`
window (bases)	`100`
minimum length (bases)	`1`
probe	`32 bases, errors 0.2, terminal window 60 bases`
threads	`4`

Fig. 1: The head of the report: the logo, the time of writing and the run parameters.

Groups.
With several groups of input files (*Outputs*), a table of the groups, with their reads, the reads written, those with both primers and, with references, the internally primed read ends; the names of the groups link to their sections.



Groups

group	files	reads	written	both primers
barcode01	1	6	100.0%	100.0%
barcode02	1	6	83.3%	66.7%

Fig. 2: The groups of the test data `testdata/fastq_pass` (two barcode directories), without references.

Then, for all reads and for each group:

Reads at each step.
The reads read, those with the primers of both ends, of one end or of none, the primer dimers, the reads too short to be written, the reads written and those found on the minus strand, as tiles and as bars. Many reads without primers mean a scheme which does not match the primers of the run (`-s`), reads already trimmed, or a higher error rate than `-e` allows.

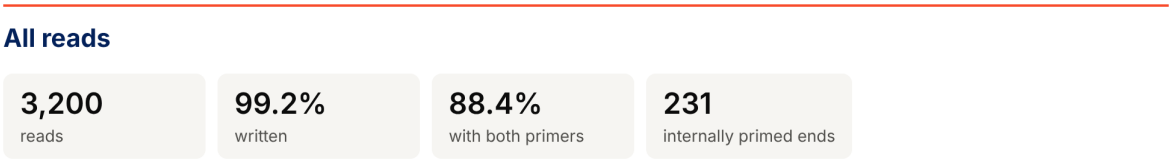


Fig. 3: The tiles of all reads: 3,200 reads, 99.2% written, 88.4% with both primers, 231 internally primed read ends.

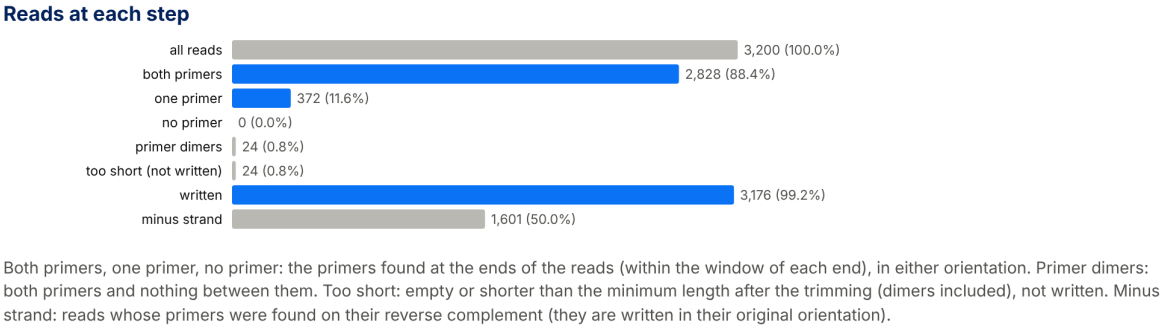


Fig. 4: All reads: 99.2% of the reads were written; 88.4% had both primers, 11.6% only one (the simulation breaks some reads); the 24 primer dimers were not written.

Primers found.
The reads with the primer of each end, by primer of the scheme (e.g. the two forms of the Uni12 primer of the `mbt` scheme).

Internal priming.
With references: the read ends of each end by origin (terminal, internal, not located), and the internal priming sites, by decreasing number of reads: the reference, the end, the position of the first base after the primer (1-based on the plus strand of the reference, as in the `ml`/`mr` tags of the reads) and the range of the positions of the site (the read ends of a site map a few bases apart, because of the sequencing errors: positions at most 5 bases apart are one site). Without references, a note: the sites are not known, and the length check below is the check of internal priming.

Internal priming

end	terminal	internal	not located
left	2,697	231	36
right	3,034	0	30

reference	end	position	range	reads	% of the reads
segment 4 (HA), KF918483.1	left	1090	1089-1092	224	7.0%
segment 4 (HA), KF918483.1	left	1158	1157-1158	6	0.2%
segment 1 (PB2), NC_002023.1	left	1595	–	1	0.0%

Read ends classified as internally primed: the bases after the primer were located at an interior position of a reference. Position: that of the first base after the primer (1-based, on the plus strand of the reference, as in the mI/mr tags of the reads); the positions at most 5 bases apart are one site (range), the position given is that of most reads. Percentages of all reads. Sites shown: 3 of 3 (at most 20).

Fig. 5: Internal priming in the simulated reads: 231 left ends were classified as internal; 224 of them are the site of the simulation, in the HA (KF918483.1), the first base after the primer at 1090.

Complete amplicons.

For each reference (or for all reads, without references): the complete amplicons (both primers found), the most frequent length of the template between the primers, and the amplicons shorter than 90% of the reference, with their most frequent length; then the histograms of the lengths, as in `lengths.tsv`. Internal priming, and deletions, show as short amplicons.

Complete amplicons

reference	length	complete amplicons	most at	short	short, most at
segment 7 (M), NC_002016.1	1,027	353	1000-1009	0 (0.0%)	–
segment 6 (NA), NC_002018.1	1,413	349	1380-1389	0 (0.0%)	–
segment 5 (NP), NC_002019.1	1,565	355	1540-1549	0 (0.0%)	–
segment 8 (NS), NC_002020.1	890	344	860-869	0 (0.0%)	–
segment 2 (PB1), NC_002021.1	2,341	340	2310-2319	0 (0.0%)	–
segment 3 (PA), NC_002022.1	2,233	341	2200-2209	0 (0.0%)	–
segment 1 (PB2), NC_002023.1	2,341	354	2310-2319	2 (0.6%)	730-739
segment 4 (HA), KF918483.1	1,779	367	670-679	221 (60.2%)	670-679
not_located	–	1	0-9	–	–

Complete amplicons: reads with the primers of both ends; their length is that of the template between the two primers, in bins of 10 bases. With references, they are grouped by the reference to which the bases after the left primer (or else the right primer) mapped; short: shorter than 90% of the reference, from internal priming or from deletions (e.g. in defective interfering RNAs of the polymerase segments).

Fig. 6: Complete amplicons: 221 of the 367 HA amplicons are shorter than 90% of the reference, most of them 670-679 bases long, from the internal site to the end of the segment.

8.2 The JSON summary

`flucut_summary.json` contains the same information as the HTML report, in a documented format (version 1, described in `docs/json_summary.md` of the repository). Its main keys are:

- `format`, `program`, `version`, `updated`: the format version, the program and the time of writing;
- `command`, `scheme`, `trim_policy`, `references`, `settings`: the parameters of the run;
- `all`: the statistics of all reads, and `groups`: those of each group, with `reads` (the reads at each step), `primers`, `ends` and `internal_sites` (with references), `short_complete` and `complete_amplicons` (with the histograms of the lengths).

For example, the internal priming sites of each group with `jq`:

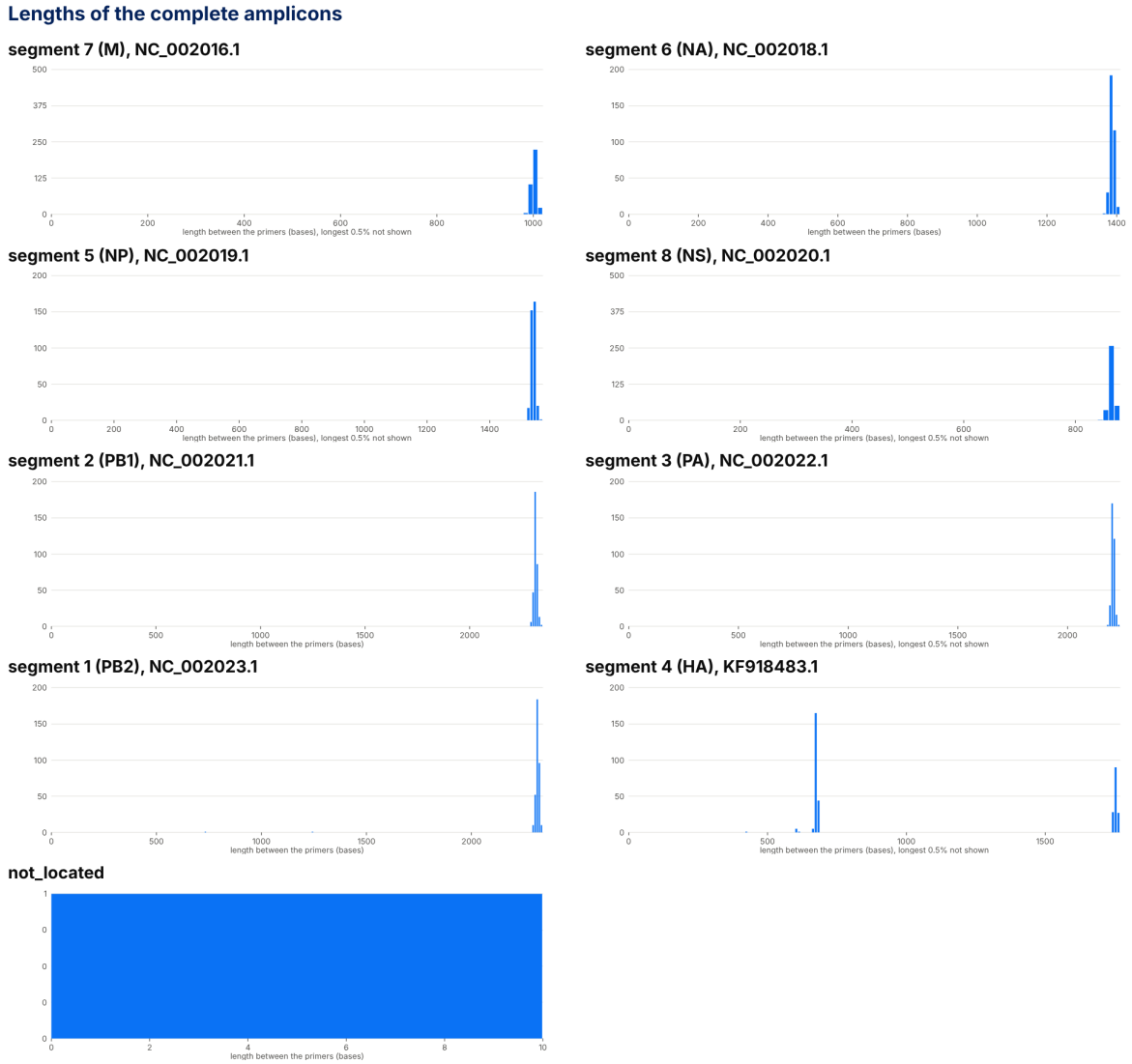


Fig. 7: The histograms of the lengths of the complete amplicons, one for each reference and one for the amplicon whose read ends were not located; that of the HA (KF918483.1) has two peaks, the amplicons started at the internal site and the complete segment.

```
jq -r '.groups[] | .name as $g | .internal_sites[]? |
       [$g, .reference, .end, .position, .reads] | @tsv' \
    out/flucut_summary.json
```

which gives for the simulated reads:

```
reads  KF918483.1   left  1090  224
reads  KF918483.1   left  1158  6
reads  NC_002023.1  left  1595  1
```

9 Live mode

With `--live`, flucut follows a MinKNOW run while it is sequencing: the files of reads are trimmed as they are written, and the outputs, the tables and the reports are updated after each file, so that the samples can be assembled, or at least looked at, before the end of the run. The live mode is that of Lumipore (its real-time mode, `--watch`).

```
flucut --live -o out /data/run1/fastq_pass
```

The inputs must be directories: typically the `fastq_pass` directory of the run, whose `barcode` subdirectories are the samples (also those created during the run), or a directory of files of one sample.

9.1 How the files are taken

The input directories are scanned every `--scan-interval` seconds (default 10). A new file is trimmed when it is ready: not modified for `--min-age` (default 60 s; e.g. `30s`, `2m`) and complete (a gzipped file whose stream is not truncated, a FASTQ file whose last line is complete, a BAM file ending with its end-of-file marker). Its reads are appended to the output of its sample (`<sample>.fastq.gz`: gzip files can be concatenated), and one line per file reports them, e.g.:

```
SUCCESS | barcode01: FBA12345_pass_barcode01_0.fastq.gz: 3912 of 4000
  reads written (3801 with both primers, ...)
```

A file which changes after it was trimmed is reported (`modified after being processed`): its new reads are not trimmed; `--min-age` should then be increased.

`-e auto` (the default) is decided at the first file ready, from its first 1000 reads, as in a normal run (*Command line reference*).

9.2 The reports

After each file, `summary.tsv`, `lengths.tsv`, the JSON summary and the HTML report are written again (each to a temporary file, then renamed, so that a reader never sees an incomplete one). While the run is in progress, the HTML report says so and reloads itself every 30 seconds in the browser, and the JSON summary has `"running": true`. At the end, the final versions are written, and the counts of each sample are reported as in a normal run, with the warnings of the lengths.

9.3 The end

The live mode ends:

- at the end of the run: when MinKNOW writes its final summary (`final_summary_*.txt`) in an input directory or in its parent (the run directory of `fastq_pass`); the files present are then trimmed without waiting for their minimum age;
- after `--timeout` without new files (e.g. `30m`, `2h`; a number is in minutes; default: none);
- with Ctrl-C (or SIGTERM): the file being trimmed is completed, then the final reports are written; a second Ctrl-C stops flucut at once.

9.4 Resuming

The list of the files trimmed for each sample and its counts are saved in the output directory (`<sample>.flucut-files`, `<sample>.flucut-counts`) after each file. With `--resume`, a live run started again with the same output directory skips the files already trimmed, and continues the outputs and the counts: e.g. after a restart of the computer. Without `--resume`, the outputs of each sample are written again from the beginning.

The outputs of a live run are the same as those of a normal run on the files of the end of the run (`tests/test-live.sh` checks it, also after an interruption and a resumption).

10 Using the reads with IRMA

The trimmed reads of each sample are the input of IRMA, e.g. with the configuration for Nanopore reads:

```
IRMA FLU-minion trimmed/barcode01.fastq.gz sample01
```

10.1 What was checked

These points were measured with IRMA 1.4.0 (FLU-minion) on simulated reads (`benchmark/RESULTS.md`):

- **The tags.** IRMA accepts the tags in the titles: the consensus was the same with and without them. `--no-tags` writes the titles unchanged, if another program needs it.
- **The ends of the consensus.** With FLU-minion as it is, IRMA does not extend the consensus beyond its own references: 20 to 58 bases are missing at each end of the segments, with or without trimming. With the reference elongation (an external configuration file with `SKIP_E=0`, given with `-c`), the consensus of the reads trimmed by flucut ends exactly where the primers began: 12 bases short at one end and 13 at the other, without errors. With untrimmed reads, the consensus extends into the flanks, with insertions.
- **Keeping the termini.** With `--trim terminus` and the elongation, the consensus has complete ends, but the bases there are the primer's: the fourth base is an ambiguity (R) in all segments, and where a segment differs from the primer, the primer's base is kept.
- **Low coverage.** FLU-minion replaces with N the bases supported by fewer than 50 reads (`MIN_CONS_SUPPORT`) in the amended consensus: with fewer reads, the amended consensus is all N, whatever the trimming; the majority consensus (`A_<gene>.fasta`) is still written.

The configuration file for the elongation:

```
echo 'SKIP_E=0' > elong.sh
IRMA FLU-minion trimmed/barcode01.fastq.gz sample01 -c elong.sh
```

11

Performance and accuracy

The numbers of this chapter come from the benchmark of the repository (`benchmark/`): what was run, on which data, with which versions and commands, is in `benchmark/RESULTS.md`; the scripts are in `benchmark/`. The reads are simulated, so that the origin of every base is known: amplicons of a genome made of the segments of A/Puerto Rico/8/1934 (H1N1) and the HA of A/chicken/Cambodia/X0815302/2013 (H5N1), which carries the internal priming site of the I348K artefact, with Nanopore-like errors (`benchmark/README.md`). No real run has been measured yet.

11.1 The consensus

On 3200 simulated reads (400 per segment, 60% of the HA reads started at the internal site), assembled by IRMA 1.4.0 (FLU-minion):

- the untrimmed reads give the artefact: AAA (lysine) at the I348K codon of the majority consensus; the reads trimmed by flucut give ATA, the truth;
- with the reference elongation of IRMA, the consensus of the reads trimmed by flucut has no error, and ends 12 and 13 bases short of the segments (the primers); with `--trim terminus` it has complete ends, with the primer's bases there (*Using the reads with IRMA*);
- at 10, 20 and 40 reads per segment (three simulations each), the untrimmed reads give the artefact in 5 of 9 runs, the reads trimmed by flucut in none.

11.2 The reads

Bases of the 3200 reads by their true origin, after the trimming (flucut of the current version; the consensus above was made with the reads of the previous one, which removed 882 template bases, kept 1852 primer bases and 7557 flank bases):

tool	template removed	primer kept	flank kept	internal site kept
untrimmed	0	102554	127032	2698
flucut	680	1819	7542	23
cutadapt	102408	3037	8784	25

cutadapt (`-g` and `-a`, not anchored, as the reads have flanking bases) removes everything before a primer-like sequence found anywhere in a read: with 2 errors, Uni12 also matches inside the segments, e.g. at the

I348K site, and complete HA reads are cut there. Its consensus is nonetheless the same as that of flucut in these simulations, also at low coverage. The comparison with the other tools (IRMA-core trimmer, BBDuk, Porechop_ABI, dorado, Barbell, samtools ampliconclip) is in `benchmark/RESULTS.md`.

11.3 Reads with barcodes

On 3200 reads simulated with the native barcoding adapters of Oxford Nanopore (NB01-NB04) outside the primers:

- the untrimmed reads gave the right codon at the I348K site, but only by 158 reads against 152: the aligner of IRMA clips more of the start of the internally primed reads after an adapter. The artefact thus depends on the library and on chance; after flucut, 157 reads against 1;
- with the elongation, the consensus of the reads trimmed by flucut has no error, as with cutadapt, dorado, samtools ampliconclip and Barbell;
- without the adapter search (`--adapters none`), flucut removed 254 template bases and left 2221 primer bases and 28539 bases of adapters, at the read ends where it found no primer; with it (the default), 267, 1019 and 698 (Barbell: 1883, 445 and 4375; dorado: 3714, 6452 and 360). The consensus is the same. The reads without barcodes are trimmed as without the search.

11.4 Speed

flucut trimmed the 3200 reads (5.3 million bases) in 0.7 seconds with one thread. The classification against references, through the k-mer index, adds little: 0.8 s instead of 0.7 s with 8 references, 1.1 s with 80 (measured before the search of the truncated primers; with the exhaustive search, 2.0 and 14-16 s).

With threads: the same reads ten times (32,000 reads), with the 8 references, on a computer with 4 cores (three runs each): 6.7-6.9 s with `-j 1`, 2.1-2.5 s with `-j 4` (3.0-3.2 s with the threads of the previous version, started for each batch); the output files are identical.

12 Troubleshooting

Each message of flucut, its cause and what to do. Errors end the program with exit status 1.

12.1 Errors in the command line

`--trim terminus requires --references`
: Keeping the termini without references would keep the primer bases of the internally primed reads (the artefact). Give the references (`-r`), or use the default `--trim primer`.

`unknown scheme '<name>'`
: The name given to `-s` is not a built-in scheme; the help lists them.

`'primer' or 'terminus' expected`
: The value of `-T` is neither.

`a fraction below 1, a whole number of bases or auto expected`
: The value of `-e` is negative, not a number nor `auto`, or a fraction above 1 (e.g. 1.5).

`argument must be an integer ...`
: A value of `-w`, `-m` or `-j` out of its range.

`Option '--probe-len' / '--probe-errors' / '--terminal-window' ...`
: A value out of its range (see *Command line reference*).

`the output directory is empty`
: `-o ""` was given.

12.2 Errors in the inputs

`<file>:  not a file of reads`
: An input which is not a directory must end in `.fastq`, `.fq` (optionally `.gz`) or `.bam`.

`No FASTQ or BAM files in the input group <name>` (warning)
: A directory given as input, or one of its barcode directories, contains no file with these suffixes: its output is empty.

`The input directory <dir> contains barcode directories and <n> FASTQ or BAM file(s) outside them` (warning)
: Only the barcode directories are trimmed; the files beside them are not read.

The input group name <name> is not unique
: Two groups have the same name (e.g. two directories `barcode01` in different runs, or a file and a directory with the same name): trim them in separate runs, with different output directories.

<file>: the input would be overwritten by the output
: The output directory is the directory of an input with the same name: choose another output directory.

Errors in the references file
: `the sequence of "<name>" is empty`, `no sequence found`, `invalid base`, `sequence before the first header line`: the file given to `-r` is not a valid FASTA file.

12.3 Messages about the reads

not located in the references: <n> left, <n> right
: Read ends whose probe was not found in the references. A few are expected (reads with many errors, dimers); many mean that the references do not fit the run (e.g. a missing subtype): download references of the subtypes of the run.

<reference>: <n> of <m> complete amplicons shorter than 90% of the reference
: Complete amplicons much shorter than their segment: internal priming or deletions (*Internal priming*). Look at the reads of `lengths.tsv` and at the `ml`/`mr` tags.

Many reads without primers (no_end in summary.tsv)
: The scheme does not match the primers of the run (`-s`), the reads were already trimmed (e.g. by the basecaller), or the error rate of the reads is higher than `-e` allows (with `-e auto`, see the rate measured in the report).

13

Command line reference

```
flucut [options] <input>...
```

13.1 Arguments

<input>
: A FASTQ file (optionally gzipped), an unaligned BAM file, or a directory of such files (recognised by the suffixes `.fastq`, `.fq`, optionally followed by `.gz`, and `.bam`; e.g. a barcode directory of a MinKNOW run, whose files are trimmed into one output). A directory with subdirectories whose name starts with `barcode` (e.g. the `fastq_pass` directory of a MinKNOW run) gives one output for each of them, and the files beside them are not read. Each group of files gives one output (*Outputs*).

13.2 Options

-o, --outdir <dir>
: The output directory, created if needed. Default: `flucut-output`.

-s, --scheme <name>
: The primer scheme (*The primers*): `uni12-mbt13` (default), `mbt` or `uni`.

-T, --trim <policy>
: `primer` (default): remove every primer-derived base. `terminus`: keep the conserved termini; requires `--references` (*How it works*).

-r, --references <file>
: A FASTA file of segment references: the read ends are classified as terminal or internally primed (*Internal priming*). With `--trim primer` the classification is only reported; with `--trim terminus` it decides the trimming of the internal ends. The messages and reports name a reference with its segment when its description (the header after the name) says it: `segment <n>` (as in the NCBI records), or a gene or protein name (e.g. `HA`, `hemagglutinin`, or the IRMA names like `A_HA_H5`). Default: none.

--adapters <kits>
: The sequencing adapters of Oxford Nanopore searched at the read ends at which no primer is found (*How it works*): a comma-separated list of `nb` (native barcoding), `rbk` (rapid barcoding) and `lsk` (ligation, without barcodes), or `none`. Default: `nb,rbk,lsk`.

-e, --max-errors <x>
: The errors allowed in a primer: a whole number of bases, below 1 a fraction of the length of the primer, or `auto`: decided from the error rate of the primer alignments in the first 1000 reads of the first input file (with 0.2): 0.2 (2 errors in Uni12), or 0.3 (3 errors) if that rate is at least 4% (*How it works*). The value decided and the rate measured are in the messages (`-v`) and in the reports. Default: `auto`.

-w, --window <n>
: A primer is accepted at an end only if it begins (left end) or ends (right end) within <n> bases of that end of the read. Default: 100.

-m, --min-length <n>
: Reads shorter than <n> bases after the trimming are not written. Default: 1 (empty reads, e.g. primer dimers, are never written).

-j, --threads <n>
: The number of worker threads (1 to 256; default 4), each trimming batches of 1000 reads (*How it works*). The output does not depend on it. Default: 4.

--no-tags
: Do not append the tags to the read titles.

13.3 Classification against the references

These options are used with `--references`.

--probe-len <n>
: The number of bases after the primer of an end located in the references. At least 16. Default: 32.

--probe-errors <x>
: The fraction of the probe allowed to be errors (0 to below 1). Default: 0.2.

--terminal-window <n>
: A probe mapped within <n> bases of an end of a segment is terminal. Default: 60.

--exhaustive-search
: Align each probe to the whole of every reference, without the k-mer index (slower; for checking the index).

13.4 Live mode

The live mode (*Live mode*): its options can only be given with `--live`.

--live
: Trim the files of the input directories as they are written (e.g. by MinKNOW), appending to the outputs and updating the tables and reports after each file; stop at the end of the run (a file `final_summary_*.txt` in an input directory or in its parent), after `--timeout`, or with Ctrl-C. The inputs must be directories.

--timeout <t>
: Also stop after <t> without new files (e.g. `30s`, `60m`, `2h`; a number: minutes). Default: none.

--min-age <t>
: Trim a file only <t> after its last change (e.g. `30s`, `2m`; a number: seconds). Default: 60 seconds.

--scan-interval <s>
: Seconds between the scans of the input directories (at most 3600). Default: 10.

--resume
: Skip the files already trimmed by an interrupted live run with the same output directory, continuing its outputs and counts.

13.5 Other options

-q, --quiet
: Only error messages.

-v, --verbose
: Additional information (e.g. the values of the options).

-D, --debug
: Debugging information.

-L, --logfile <file>
: The messages to a file instead of the standard error.

-h, --help
: The help, with the list of the schemes.

-V, --version
: The version.

Only one of `-q`, `-v` and `-D` can be given. Invalid values of the options are errors (exit status 1), reported with the usage.

The messages go to the standard error; in a terminal they are colored by their kind:

- `SUCCESS` (green): the reads written for each output, and the reports at the end;
- `WARNING` (magenta): something to look at, e.g. complete amplicons shorter than the closest reference (below 90% of its length: internal priming or deletions; with `-r`), named by their segment when the reference says it, as in `HA segment: 4 of 9 complete amplicons shorter than the closest reference (NC_004908.1, 1714 bases)`, or files of reads ignored beside the barcode directories;
- `ERROR` (red): the program stops;
- with `-v`, also `PARAM` (blue: the values of the options), `PHASE` (bright yellow: the steps, e.g. the trimming of each file) and `INFO` (yellow: the scheme and its primers, the input groups, the references read, the internally primed ends, the lengths of the amplicons without references, the tables written).

Without `-v`, a run thus prints one green line for each output, a warning if any, and a green line for the reports.

14 Glossary

amplicon
: A molecule made by the PCR: here a segment, or part of it, between the two primers.

complete amplicon
: A read in which the primers of both ends were found.

conserved terminus
: The 12 or 13 bases at the ends of the influenza A segments, the same in all segments and subtypes, to which the universal primers bind.

flank
: The bases of a read outside the primers: adapter, barcode, and other bases of the library preparation.

internal priming
: The start of an amplicon by a primer annealed at an interior position of a segment, whose bases then replace those of the template in the reads.

probe
: The bases of a read after the primer of an end, located in the references to classify the end.

scheme
: The primers expected at the two ends of an amplicon.

tail
: The bases of a primer before its conserved terminus, added by the protocol (e.g. `ACGCGTGATCAG` of the mbtuni primers).

window
: The part of a read, at each end, in which the primer of that end is accepted (`-w`).

15 About

flucut is developed by Giorgio Gonnella at BAIA, Institut Pasteur du Cambodge. It is free software under the ISC license (`LICENSE`).

The source code: https://github.com/baia-ipc/flucut

flucut includes libcbaia (ISC), edlib (MIT), libyaml (MIT) and zlib (zlib license), in `ext`, each with its license.

15.1 Use of AI tools

The code, the documentation, the benchmark and this manual were written with the help of an AI assistant, under the direction and review of the author. The logo and the illustration of the title page were made with AI image tools.